

FalconScope: Effective and Efficient Detection of Hidden Web Interfaces in IoT Devices

Jiaming Guo

Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China
guojiaming@iie.ac.cn

Haoran Yang

Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China
yanghaoran@iie.ac.cn

Kuihao Yan

Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China
yankuihao@iie.ac.cn

Jiekang Hu

Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China
hujiekang@iie.ac.cn

Xiaoqi Jia*

Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China
jiaxiaoqi@iie.ac.cn

Haichao Du

Qihang Zhou
Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China
duhaichao@iie.ac.cn
zhouqihang@iie.ac.cn

Abstract

Hidden web interfaces in Internet of Things (IoT) devices pose significant security threats by unintentionally exposing inadequately protected functionalities, enabling attackers to bypass authentication, alter configurations, leak sensitive data, or execute arbitrary commands. Despite recent advancements, current detection approaches suffer from two critical challenges: 1) inadequately model the complex internal routing mechanisms of IoT firmware, leading to incomplete interface enumeration and substantial false negatives; and 2) inefficiently generate probing requests and verify unauthorized access due to limited semantic understanding of interface communication protocols. To overcome these challenges, we introduce FALCONSCOPE, a novel system combining precise firmware routing modeling and Large Language Model (LLM)-driven semantic analysis to detect hidden web interfaces effectively and efficiently. FALCONSCOPE achieves this through two key innovations: 1) a static analysis technique precisely reconstructs the device's internal routing mechanisms, enabling comprehensive enumeration of Routing Unique Identifiers and their corresponding backend handlers; and 2) an LLM-powered semantic engine automatically generates syntactically and semantically valid HTTP requests to efficiently trigger backend logic, coupled with semantic validation of device responses to accurately confirm unauthorized access. Evaluations on 11 real-world IoT devices from four major vendors demonstrate

that FALCONSCOPE significantly surpasses existing state-of-the-art tools, detecting 620 hidden web interfaces—103 times more than IoTScope—while consuming only 3.6% of its analysis time. Following responsible disclosure, 50 issues have been assigned CVE IDs.

CCS Concepts

• Security and privacy → Web application security.

Keywords

Vulnerability Detection; Web security; Internet of Things; Large Language Model

ACM Reference Format:

Jiaming Guo, Haoran Yang, Kuihao Yan, Jiekang Hu, Xiaoqi Jia, Haichao Du, and Qihang Zhou. 2026. FalconScope: Effective and Efficient Detection of Hidden Web Interfaces in IoT Devices. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792431>

1 Introduction

Internet of Things (IoT) devices have become integral to modern life, with a significant increase in the number of connected devices in recent years. Predictions indicate that by the end of 2030, the global number of connected IoT devices will surge to approximately 40 billion [2]. Despite their convenience and widespread adoption, IoT devices pose severe security risks. Reports indicate that over half of IoT devices are vulnerable to medium or high-severity attacks, presenting substantial threats to user privacy and system security [34]. Among various attack vectors, hidden web interfaces have recently emerged as a particularly severe yet easily overlooked threat (We use hidden interface as shorthand in the remainder of the paper). These interfaces, often concealed within web-based management interfaces of devices intended for configuration convenience, unintentionally expose numerous inadequately protected

*Corresponding Author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
WWW '26, Dubai, United Arab Emirates.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792431>

functionalities. Attackers can exploit these hidden interfaces to bypass authentication, modify device configurations illegally, extract sensitive information, or even execute arbitrary code. For instance, CVE-2024-13030 [24] reveals attackers can configure critical functions such as firewall rules via specially crafted HTTP POST requests without authorization. OWASP [27] has classified Broken Access Control as one of the most critical web application security risks, and hidden interfaces embody this risk within IoT devices.

Unfortunately, existing IoT firmware analysis and vulnerability detection methods are insufficient for effectively uncovering such vulnerabilities. For example, IoTScope [37] employs URL enumeration but neglects the semantic information embedded in requests, resulting in numerous missed detections and an inability to effectively identify interfaces that require specific parameters to trigger backend function execution. Existing methods face two primary challenges in detecting hidden interfaces: **C1. Incomplete Interface Discovery:** Current approaches fail to accurately model the complex internal routing mechanisms of IoT devices, particularly how external requests map to specific backend handlers. This oversight prevents systematic enumeration of all potential interface entry points, particularly those interfaces not directly linked to explicit file paths, resulting in significant false negatives. **C2. Inefficient Triggering and Verification:** Existing methods lack the semantic understanding for constructing requests that accurately trigger backend logic. They rely on random mutations (fuzz testing) or simple URL probing, unable to generate valid requests that match backend processor expectations. Additionally, they inadequately interpret responses, limiting the accurate verification of hidden interfaces.

To address these challenges, we propose FALCONSCOPE, a novel solution designed to efficiently and accurately detect hidden interfaces in IoT devices. The key insight guiding FALCONSCOPE is that IoT device backends contain numerous hidden interfaces governed by an internal routing mechanism. This mechanism parses a Routing Unique Identifier (RUI) embedded in incoming requests to distribute them to corresponding backend handlers. Thus, accurately identifying hidden interfaces hinges on: **A1. Precisely modeling and exhaustively enumerating the firmware routing mechanism through static analysis, constructing a comprehensive routing map from RUIs to backend handlers. A2. Leveraging the semantic understanding and generative capabilities of large language models (LLMs) to automatically generate syntactically and semantically valid HTTP probing requests, enabling efficient interface activation and accurate verification of unauthorized access through semantic analysis of device responses.**

Building on this methodology, we implemented a prototype of FALCONSCOPE, comprising three core components: the Interface Enumerator (static analysis to pinpoint routing mechanisms and handlers), the Semantically-Enhanced Request Generator (LLM-driven generation of valid requests), and the Unauthorized Access Verifier (LLM-driven response validation). Evaluations on 11 physical IoT devices from four major vendors demonstrate FALCONSCOPE's superior performance, successfully identifying 620 hidden interfaces, which is 103 times more than IoTScope, while using only 3.6% of IoTScope's analysis time, achieving an average recall of 98.6%, and reducing the probing request volume by approximately 97.6%. The discovered interfaces have significant security impacts, resulting in the responsible disclosure of 50 CVE identifiers.

In summary, our primary contributions are:

- Identifying and articulating two critical challenges of existing hidden interface detection methods in IoT: incomplete interface discovery due to inadequate modeling of routing mechanisms, and inefficient triggering and verification stemming from limited semantic understanding.
- Proposing a novel methodology that combines precise firmware routing modeling with LLM-driven semantic request generation and response validation to address these challenges. We designed and implemented this methodology in the FALCONSCOPE system. We make the data and source code available on the FALCONSCOPE Site to facilitate future research: <https://sites.google.com/view/falconscope>.
- Conducting extensive evaluations on 11 real-world commercial IoT devices from diverse vendors. Results highlight FALCONSCOPE's superior detection efficiency and coverage (620 hidden interfaces detected, 50 CVEs obtained), demonstrating the effectiveness and efficiency of our approach.

2 Background and Motivation

2.1 Web Services in IoT Devices

Modern IoT devices utilize embedded web services for user-based configuration and management. These services consist of a front-end (HTML, JavaScript) that translates user actions into HTTP requests, and a backend that processes these requests. The backend, typically implemented as binaries (e.g., httpd daemons, CGI scripts), executes core device functionalities such as retrieving state, updating configurations, or performing actions. To handle the diverse range of incoming requests, the backend employs an internal routing mechanism to dispatch each request to its appropriate handler.

2.2 Routing Mechanism and Hidden Interfaces

Terminology. In this paper, we define a hidden web interface as a backend handler/end-point in an embedded web management service that is reachable by an unauthenticated request due to Broken Access Control (i.e., bypassing the intended authentication workflow). For brevity, we refer to hidden web interfaces as hidden interfaces in the remainder of the paper.

The concept of routing mechanisms is widely employed in modern web frameworks [5, 6, 33] for dispatching HTTP requests. This established practice inspired our investigation into analogous mechanisms within IoT devices. Through a large-scale empirical study of 120 firmware images (detailed in Table 5), we confirmed that IoT web services are similarly governed by internal routing maps embedded within the firmware. This routing map specifies rules for parsing an incoming HTTP request and dispatching it to the appropriate backend handler. Specifically, the backend extracts key information (e.g., HTTP method, URL, headers, body parameters) and uses it to look up a specific identifier—which we term a Routing Unique Identifier (RUI)—in the map. Upon a successful match, the request is dispatched to its corresponding handler, which may be an internal function, a server-side script, or a file access operation.

However, a critical security vulnerability arises when these handlers lack adequate authentication or authorization checks. In this case, hidden interfaces (as defined above) are reachable through an unauthorized access path—often triggered by a specific RUI—that

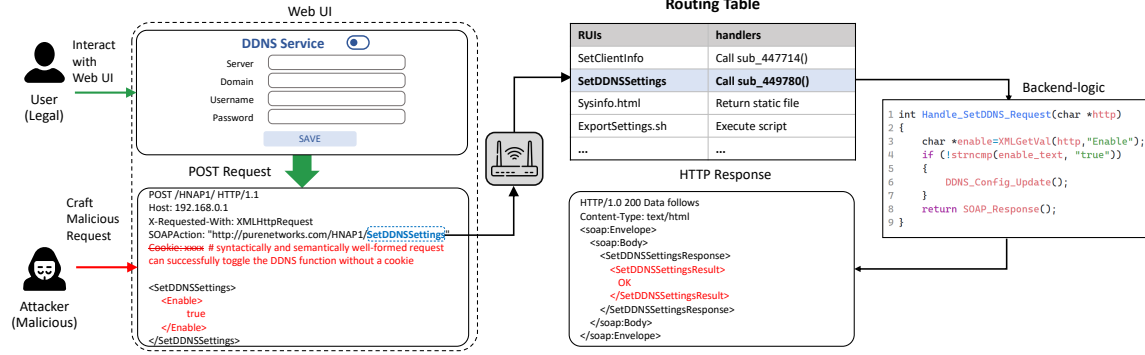


Figure 1: Motivating Example. An Unauthorized Access Vulnerability (CVE-2024-13030) Discovered by FALCONSCOPE.

bypasses the intended security workflow. Our definition aligns with IoTScope’s notion of an unauthorized interface [37], and differs from works such as EagleEye [21] that define hidden interfaces as merely undocumented. An adversary can exploit this weakness by crafting direct HTTP requests containing valid RUIs to invoke privileged functions or access sensitive resources without credentials.

While prior works have made efforts to model these routing mechanisms, significant gaps in accuracy, efficiency, and coverage remain. IoTScope [37] primarily focuses on enumerating URL-based routes, thus lacking a deep understanding of the more prevalent function-level handlers. EagleEye [21] introduces the use of LLMs to infer routing tokens from decompiled code snippets; however, its goal is to generate seeds for fuzzing undocumented interfaces, not BAC vulnerability detection. Furthermore, without the guidance of predefined routing patterns from static analysis, its LLM-based inference across a vast code space is prohibitively time-consuming. In summary, an approach that can accurately, efficiently, and comprehensively map RUIs to their backend handlers for the discovery of Broken Access Control vulnerabilities is still critically needed.

2.3 Threat Model

We focus on adversaries targeting web-based management interfaces of IoT devices to gain unauthorized access. We assume an attacker can obtain device firmware from public sources (e.g., vendor websites), decompile and analyze the embedded web-service binaries, and reach the target over a local (LAN) or wide-area (WAN) network to deliver crafted HTTP requests. The attacker aims to discover and exploit hidden interfaces—backend handlers that lack adequate authentication or authorization. Following IoTScope [37], we define hidden interfaces by their unauthorized accessibility rather than by the absence from official documentation (the latter being the focus of works such as EagleEye [21]).

2.4 Motivating Example

To illustrate the challenges in detecting hidden interfaces and motivate our approach, we present a broken access control vulnerability CVE-2024-13030 [24], which is discovered by FALCONSCOPE, as shown in Figure 1.

In a legitimate use case, an authenticated user interacts with the device’s Web UI to configure the DDNS service. Clicking the “SAVE” button triggers the front-end to generate an HTTP POST request.

This request contains the user’s configuration data and, crucially, an authentication credential (e.g., a valid session Cookie). Upon receiving this request, the backend’s internal routing table correctly maps the SetDDNSSettings RUI (found in the SOAPAction header) to its designated handler, sub_449780() (hereafter Handle_SetDDNS_Request()), which updates the device’s configuration and returns a success response to the user. However, a critical flaw exists in the backend implementation. As shown in the Backend-logic code snippet, the Handle_SetDDNS_Request() function directly proceeds to parse the request and update the DDNS configuration (DDNS_Config_Update()) without performing any authentication or authorization checks. The logic only verifies the presence and value of the <Enable> tag; it never validates if the request originated from an authenticated session. This oversight creates a hidden, unauthorized access path to a privileged function. An unauthenticated attacker can exploit this vulnerability by completely bypassing the Web UI. The attacker crafts a malicious HTTP POST request that mimics the structure of the legitimate one, including the correct SetDDNSSettings RUI in the SOAPAction header and the required XML payload. However, the attacker’s request deliberately omits the Cookie header or any other authentication token. Despite the lack of credentials, the vulnerable backend still accepts and processes the request, allowing the attacker to remotely enable or disable the DDNS service at will.

This example highlights the core challenges that FALCONSCOPE is designed to address. Discovering such a vulnerability requires: (1) accurately identifying the SetDDNSSettings RUI and its mapping to the handler sub_449780() from the firmware binary; (2) understanding the semantic requirements of the request, such as the need for an <Enable> tag in an XML payload; and (3) correctly interpreting the “OK” response as a confirmation of a successful, unauthorized state change. Simple URL enumeration or crash-based fuzzing would almost certainly fail to uncover this type of logic-based, hidden interface vulnerability. Unfortunately, state-of-the-art tools fail to discover such vulnerabilities. IoTScope’s URL-based route enumeration overlooks critical function-level handlers like the one in our example, and its keyword-based bug oracle cannot comprehend the semantic meaning of the device’s response. EagleEye also employs an LLM, but for a different and more constrained purpose: learning patterns from public interfaces to generate a dictionary of potential routing tokens for fuzzing. This approach

differs fundamentally from ours, as it aims to find undocumented interfaces rather than specifically targeting authorization flaws. Its fuzzing-based methodology is ill-suited for our goal because it primarily mutates known requests rather than synthesizing the semantically precise new requests needed to trigger complex BAC logic. Furthermore, its reliance on an initial set of requests gathered by following public documentation limits its applicability to devices without such manuals.

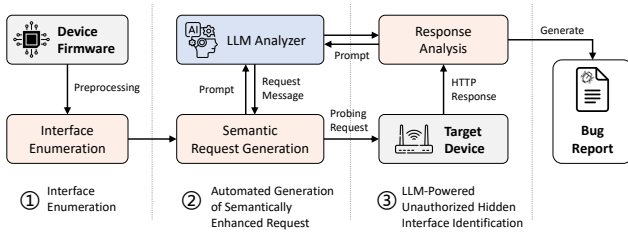


Figure 2: Workflow of FALCONSCOPE.

2.5 Challenges

Challenge 1: Incomplete Enumeration of Hidden Interfaces and Imprecise Route Mapping. IoT firmware’s intrinsic complexity and heterogeneity make it difficult to systematically enumerate hidden interfaces and accurately reconstruct their associated routing logic. Firmware typically contains numerous binaries (e.g., embedded web servers, CGI scripts) and configuration scripts that embed undocumented interface logic. Additionally, backend handlers route requests based on various Routing Unique Identifiers situated diversely in URL paths, query parameters, headers, or request bodies. The methods to map these identifiers to corresponding handlers vary widely—ranging from explicit function registration and static lookup tables to implicit file-system-based dispatch.

Challenge 2: Inefficient and Inaccurate Triggering and Verification. Identifying potential interfaces is only the first step; meaningful vulnerability discovery requires dynamically probing them. This presents a dual challenge in both triggering the logic and verifying the outcome. First, triggering BAC vulnerabilities necessitates crafting requests that are not just syntactically correct but semantically precise. This demands exact knowledge of RUIs, their embedding positions, required parameter specifications, and diverse payload formats (e.g., Key-Value, JSON, XML/SOAP). Traditional methods like fuzzing, with their randomized inputs, consistently fail to meet these stringent semantic requirements. Second, accurately verifying a successful exploit is equally difficult. Unlike memory errors, logic vulnerabilities do not cause crashes, rendering crash-based oracles useless. The only evidence lies in the device’s response, which is often ambiguous and non-standardized. Consequently, simple heuristics based on HTTP status codes or keyword matching are notoriously unreliable, leading to high rates of both false positives and negatives. Together, these issues demand a new approach that can intelligently synthesize semantically valid requests and perform a deep, context-aware analysis of their responses to reliably confirm exploitation.

3 Design

As illustrated in Figure 2, the overall workflow of FALCONSCOPE takes an IoT device’s firmware image as input and outputs alerts for hidden interfaces that are vulnerable to unauthorized access. FALCONSCOPE first performs static analysis on the firmware to systematically enumerate all potential hidden interfaces. It then leverages Large Language Models (LLMs), enhanced with few-shot learning and Chain-of-Thought [36] (CoT) prompting, to automatically generate syntactically correct and semantically valid HTTP request messages, intentionally crafted without authentication-related fields such as cookie or authentication headers, to trigger the processing logic of these hidden interfaces. Finally, FALCONSCOPE performs deep semantic analysis of the device responses to accurately detect unauthorized hidden interfaces.

3.1 Interface Enumeration

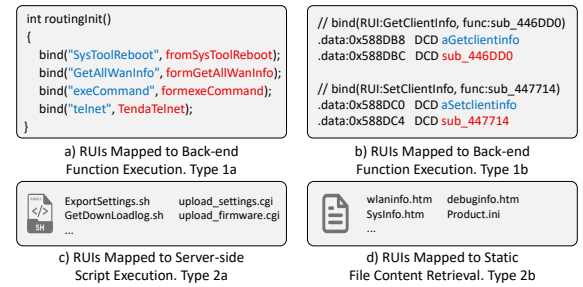


Figure 3: Handlers for Routing Unique Identifiers.

As previously described, the backend logic of an IoT device maintains an internal routing map that associates user-supplied *Routing Unique Identifiers (RUIs)* with their corresponding handlers. If these handlers lack sufficient protection, they become potential hidden interfaces. Therefore, the primary goal of this stage is to comprehensively and accurately enumerate all RUI-to-handler mappings within the firmware. Based on a large-scale empirical study of 120 firmware images (detailed in Table 5), we identified and categorized the prevalent RUI-to-handler mapping patterns into two main types, as illustrated in Figure 3.

Type 1: RUIs Mapped to Backend Function Execution. In this prevalent routing paradigm, the Routing Unique Identifier (RUI) typically embedded in the request URI—is used by the backend logic to locate and invoke a specific internal handler function. We identify two primary sub-types based on how this mapping is defined. In *Type 1a, Explicit Function Registration* (Figure 3a), the backend explicitly calls registration routines (e.g., `bind()`) to dynamically bind RUI strings with their corresponding handler functions at runtime. In *Type 1b, Static Data Mapping* (Figure 3b), RUIs and their associated function pointers are stored as key-value pairs within constant data segments (e.g., the `.data` section). The program then performs lookups within this static map to resolve the RUI-to-handler relationship.

Type 2: RUIs Mapped to File-based Routing. In this routing pattern, the RUI corresponds directly to a file path on the device’s filesystem, and the web server’s behavior is determined by the file type. In *Type 2a, Server-side Script Execution* (Figure 3c), the RUI maps to an executable script (e.g., shell script or CGI binary),

prompting the server to execute it. If such scripts lack proper access control, this can lead to unauthorized command execution. Conversely, in *Type 2b, Static File Content Retrieval* (Figure 3d), the RUI points to a non-executable resource (e.g., `.htm`, `.ini`), causing the server to retrieve the file's content and return it in the HTTP response. Insufficient access controls in this context can expose sensitive information, making both file-based routing patterns critical sources of hidden interfaces.

FALCONSCOPE implements these enumeration strategies in Algorithm 1 to achieve comprehensive and accurate RUI-to-handler mapping. For file-based interfaces (Type 2), FALCONSCOPE recursively scans the entire firmware filesystem and employs regular expressions to identify potential interface candidates based on common file extensions. Files ending with `.sh` or `.cgi` are flagged as potential script-based interfaces (Type 2a), whereas those ending with `.txt`, `.html`, or `.ini` are treated as potential static files (Type 2b). To further refine this large candidate set and prioritize high-value targets, FALCONSCOPE integrates a Large Language Model (LLM) for semantic filtering. The LLM summarizes each file's content and assesses its likelihood of containing sensitive information (e.g., credentials, private keys) or performing security-critical operations. This intelligent pruning process substantially reduces the subsequent analysis overhead by filtering out irrelevant artifacts such as CSS stylesheets or image files, thereby ensuring a focused and efficient enumeration process.

For function-based interfaces (Type 1), FALCONSCOPE leverages the observation that a few URI strings from the front-end often serve as entry points to discover the entire backend routing logic. It begins by extracting a limited set of URIs from the front-end and searching for their cross-references within the backend binary. The location of these references dictates the analysis strategy: If a URI is found in the `.data` section, FALCONSCOPE identifies it as a Type 1b (Static Data Mapping). It then parses the surrounding data structures to reconstruct the full key-value map of all RUIs and their handler function pointers stored in that region. If a URI appears in the `.text` section as an argument to a registration function (e.g., `bind()`), FALCONSCOPE flags this as a Type 1a (Explicit Function Registration). Instead of just analyzing this single call, it identifies the caller of this registration function—which is typically the central routing initialization function (e.g., `routingInit` in Figure 3a). By analyzing this single `routingInit` function, FALCONSCOPE can systematically extract the complete set of all dynamically registered RUI-to-handler bindings. This targeted, cross-reference-based strategy allows FALCONSCOPE to efficiently reverse-engineer the entire function-based routing configuration by using just a few initial URI seeds to pinpoint the core routing logic.

3.2 Automated Generation of Semantically Enhanced Request

Following the interface enumeration stage, FALCONSCOPE holds a comprehensive map from RUIs to backend handlers; however, RUI-to-handler mappings alone are insufficient for vulnerability testing because triggering backend logic requires complete, valid HTTP requests. The core challenge is therefore to automatically synthesize probing requests that are both syntactically correct and semantically valid for each of the hundreds of identified interfaces.

This difficulty arises from two factors: heterogeneous backend logic, which imposes distinct semantic requirements and payload formats (e.g., JSON, XML) per interface, and vendor-specific conventions for embedding RUIs (e.g., in the URI path, in custom headers such as `SOAPAction`). From a large-scale empirical analysis (Table 5) we derive two key observations: backend handler code is semantically rich—decompiled pseudocode reveals required parameter names and expected value patterns—and communication paradigms are largely consistent within device families (e.g., Tenda devices often prepend `/goform/` to RUIs while the D-Link DIR-823G places RUIs in the `SOAPAction` header and uses XML payloads), with intra-family consistency approaching 100% (Table 7).

Leveraging these insights, FALCONSCOPE adopts a semantically enhanced request-generation pipeline that fuses static analysis with the few-shot reasoning capabilities of LLMs. First, FALCONSCOPE decompiles each handler into C-like pseudocode using IDA Pro, since C pseudocode exposes parsing logic and control flow in a form that LLMs can reason about more effectively than raw assembly. Second, for each device we capture a small number of legitimate request samples via Burp Suite to provide few-shot exemplars of the device's communication paradigm, showing RUI placement, required headers, and payload structure. Note that these examples can equally be obtained from a re-hosted/emulated firmware instance. Third, FALCONSCOPE issues a Chain-of-Thought prompt to an LLM that combines the decompiled pseudocode and the few-shot samples: the LLM analyzes the handler logic to infer input semantics, aligns those semantics with the structural patterns from the samples, and synthesizes a structured HTTP request that is both syntactically and semantically valid. Finally, the system applies response-driven iterative refinement: if the device response signals a syntactic or semantic mismatch, that feedback is fed back to the LLM as additional context so it can refine subsequent generations.

For instance, to test the `SetDDNSSettings` RUI from our motivating example (Figure 1), FALCONSCOPE first decompiles its corresponding handler function. The LLM analyzes the C pseudocode and infers that the key parameter controlling the DDNS service is the value within an `<Enable>` XML tag, which should be `true` or `false`. Concurrently, by examining the few-shot samples captured from the DIR-823G, it learns that all RUIs must be placed in the `SOAPAction` header and that the payload must be XML-formatted. By combining these semantic and structural insights, FALCONSCOPE synthesizes the exact, unauthorized POST request shown in Figure 1, successfully triggering the vulnerable logic.

3.3 LLM-Powered Unauthorized Hidden Interface Identification

The final step in FalconScope's workflow is to accurately determine whether an intentionally unauthenticated request has successfully triggered a privileged operation. Traditional methods for this task, such as crash detection or keyword matching, are fundamentally inadequate for identifying the subtle logic flaws inherent in Broken Access Control vulnerabilities. Device responses are notoriously heterogeneous, and a simple keyword like "success" or an HTTP 200 status code is an unreliable indicator of a true state change. Crucially, traditional methods fail to recognize complex success indicators such as echoes confirming a state change, system command

Table 1: Dataset of device samples.

D. ID	Vendor	Series	Type	FM. Version	Binary	Arch
1	D-Link	DIR-823G	Router	V1.0.2	goahead	MIPS32
2	D-Link	DIR-618	Router	A1_FW104	boa	MIPS32
3	D-Link	DIR-605L	Router	A1_FW114	boa	MIPS32
4	D-Link	DIR-816	Router	A2_FWv1.10	goahead	MIPS32
5	D-Link	DIR-878	Router	A1_FW101	prog.cgi	MIPS32
6	Tenda	AC18	Router	V15.03.05.05	httpd	ARM32
7	Tenda	FH1202	Router	V1.2.0.14	httpd	MIPS32
8	Tenda	O1	Bridge	V1.0.0.7	httpd	MIPS32
9	Tenda	O3	Bridge	V1.0.0.10	httpd	MIPS32
10	TOTOLINK	A3700R	Router	V9.1.2u.5822	cstecgi.cgi	MIPS32
11	Netgear	R6300	Router	V1.0.1.72	httpd	ARM32

output, configuration file fragments, or data formatted as private keys, leading to high rates of both false positives and false negatives. To overcome this challenge, FALCONSCOPE introduces a novel bug oracle powered by a Large Language Model (LLM). Instead of performing simple syntactic checks, our oracle leverages the LLM’s deep semantic comprehension capabilities [8, 11, 16, 35, 38] to interpret the device’s response in the context of the request that was sent. This approach transforms the verification process from a brittle pattern-matching task into a robust semantic inference problem. The core of our method lies in a carefully designed prompt that instructs the LLM to act as a security analyst. The prompt provides the LLM with the original request’s intent (e.g., “attempting to enable the DDNS service”) and the full HTTP response from the device. The LLM is then tasked with answering a critical question: “Does this response provide evidence that the intended operation was successful?” To answer this, the LLM is guided to look for the very semantic patterns that traditional methods miss.

These indicators typically fall into two categories: 1) Unauthorized State Setting: The response contains an explicit confirmation of a state change, such as a JSON object reflecting the new configuration or a status message like `<SetDDNSSettingsResult>OK</SetDDNSSettingsResult>`. 2) Unauthorized Information Leakage: The response body contains sensitive data that should be protected, such as system command output, fragments of configuration files, or data formatted as private keys or credentials. By reasoning about the content and structure of the response, the LLM can distinguish between a genuine success confirmation and a generic or misleading message. For example, in our motivating case (Figure 1), the LLM doesn’t just see the word “OK”. It understands that the specific XML tag `<SetDDNSSettingsResult>` directly corresponds to the `SetDDNSSettings` RUI from the request, and thus interprets the “OK” within that context as a definitive confirmation of a successful, unauthorized operation. This semantic-aware approach allows FALCONSCOPE to achieve near-perfect precision, confidently identifying hidden interfaces by recognizing the rich, contextual evidence of exploitation that traditional oracles are blind to.

4 Evaluation

Implementation. We implemented a prototype of FALCONSCOPE. For firmware unpacking, we utilized binwalk [29] to extract filesystem images from commercial IoT device firmware. To automate static analysis, we developed a set of IDAPython scripts based on IDA Pro [10], enabling batch decompilation of firmware binaries

into C-like pseudocode. For LLM-assisted semantic analysis, we leveraged OpenAI’s GPT-4.1 model [26] through LangChain [19].

Experiment Settings. All experiments were conducted on a machine running on Ubuntu 20.04, equipped with Intel Core i9-10980XE CPU, 128 GB RAM. Our test dataset comprised 11 popular commercial devices acquired from 4 distinct vendors. These devices have been widely used in previous research studies. Detailed information about the devices is provided in Table 1.

SOTA Solution for Comparison. We compare FALCONSCOPE with IoTScope [37], a state-of-the-art open-source tool for detecting hidden interfaces in embedded systems. IoTScope serves as our primary baseline due to its similar goal—identifying unauthorized hidden interfaces via firmware analysis and dynamic probing—and its open-source availability, which ensures reproducibility.

4.1 Overall Results

By applying FALCONSCOPE to the devices listed in Table 1, we confirmed 620 unique, previously unknown hidden interfaces susceptible to unauthorized access. After responsible disclosure, 50 interfaces received official CVE identifiers due to their severe implications (at the time of writing), as detailed in Table 6. Specifically, 44 CVEs involved broken access control, enabling unauthorized modifications to device configurations or states; 4 pertained to information disclosure vulnerabilities exposing credentials, configuration files, or system details; and 2 were Cross-Site Request Forgery (CSRF) vulnerabilities exploitable due to missing CSRF tokens. Notably, vulnerable interfaces were not limited to standard HTTP-based web services; FALCONSCOPE also identified exploitable hidden interfaces in UPnP and HNAP protocol implementations, enabling unauthorized access to critical device functions such as DDNS settings, QoS management, and DMZ configurations. The implications of these vulnerabilities are severe, ranging from unauthorized disclosure of sensitive device data (e.g., Wi-Fi passwords, administrative credentials) to unauthorized modification of crucial configurations (e.g., firewall rules, device passwords) and control of fundamental device functionalities. Remarkably, IoTScope failed to detect any CVE vulnerabilities listed in Table 6.

True/False Positive and Negative Analysis. We performed rigorous manual verification on the hidden interfaces identified by FALCONSCOPE to assess its accuracy. Manual validation confirmed that all hidden interfaces reported by FALCONSCOPE as unauthorized were indeed true positives, resulting in zero false positives. This high precision is largely attributed to our LLM-powered verification component (Section 3.3). Unlike simple keyword matching, the LLM analyzes the semantics of device responses to identify concrete evidence of successful exploitation. Specifically, it is trained to recognize patterns such as device status change echoes, system command outputs, fragments of configuration files, or data formatted as private keys. This deep semantic understanding allows FALCONSCOPE to accurately determine whether a credential-less request has successfully triggered the intended backend handler logic. Regarding false negatives, our primary focus was on the prevalent interface paradigms discussed in Section 3.1. Consequently, it is possible that certain hidden interfaces exhibiting patterns not covered by our current analysis framework could be missed, leading to potential false negatives. We acknowledge that achieving complete

coverage of all possible interface implementations is a formidable challenge. The goal in this domain is often not to discover all hidden interfaces, but to uncover as many as possible with high confidence. We prioritize precision and depth within well-defined patterns. Expanding our analysis to include additional hidden interface patterns will be a key direction for our future research.

Table 2: Comparison between FalconScope and IoTScope. HINT represents the number of hidden interfaces. RC% is N/A when ground truth is 0.

ID	FalconScope					IoTScope				
	Req.	HINT	Vul	RC%	Time	Req.	HINT	Vul	RC%	Time
1	138	93	13	92%	264	16 704	4	4	28.6%	10 362
2	234	87	8	100%	475	4008	0	0	0	2519
3	307	74	9	100%	545	7488	0	0	0	4713
4	380	153	16	100%	525	3584	0	0	0	2325
5	376	3	1	100%	807	30 099	2	0	0	18 656
6	279	4	2	100%	416	3480	0	0	0	2620
7	327	65	9	100%	381	27 166	0	0	0	16 248
8	226	2	0	N/A	288	3584	0	0	N/A	2294
9	109	2	0	N/A	90	1938	0	0	N/A	1286
10	232	137	10	100%	349	10 323	0	0	0	61 446
11	793	0	0	N/A	1060	31 801	0	0	N/A	21 581
Total	3401	620	68	N/A	5200	140 175	6	4	N/A	144 050
Avg.	309.18	56.36	6.18	98.6%	472.73	12 743.18	0.55	0.36	5.8%	13 095.45

4.2 Comparison with State-of-the-Art

We evaluated FALCONSCOPE’s ability to detect unauthorized hidden interfaces by benchmarking it against IoTScope, a state-of-the-art open-source tool, using the device dataset from Table 1. To facilitate a comprehensive comparison, we constructed a ground truth set for each device by aggregating all publicly known vulnerabilities (n-days) and all unique vulnerabilities discovered by both tools (0-days). A detailed comparison is presented in Table 2.

Effectiveness. In terms of effectiveness, the results summarized in Table 2 clearly illustrate FALCONSCOPE’s significant advantages over IoTScope. Across the 11 devices, FALCONSCOPE achieved an impressive average recall of 98.6%, successfully identifying 68 out of 69 known vulnerabilities in the ground truth. This high recall led to the discovery of 620 unique unauthorized hidden interfaces and the assignment of 50 new CVE IDs. In stark contrast, IoTScope detected only 6 hidden interfaces in total, resulting in a mere 4 confirmed vulnerabilities and an average recall near zero. This considerable disparity in detection capability primarily arises from two key methodological advancements: First, FALCONSCOPE employs a more precise and comprehensive interface enumeration Algorithm 1. By accurately reconstructing the internal routing mechanisms through static analysis, FALCONSCOPE systematically enumerates all potential handlers. This enables coverage of numerous interfaces that IoTScope, reliant on speculative file-path-based URL enumeration, inevitably overlooks. As shown in Table 2, this superior coverage is a direct contributor to FALCONSCOPE’s high recall, allowing it to find vulnerabilities in devices where IoTScope found none. Second, FALCONSCOPE leverages semantic-driven vulnerability verification using LLMs (Section 3.3). Unlike IoTScope, which relies on predefined keyword matching, FALCONSCOPE can semantically comprehend diverse and complex device responses (e.g., Base64-encoded credential leaks, XML-structured state changes).

This significantly improves its ability to confirm successful exploitation attempts, thereby reducing false negatives and further boosting its recall. For instance, on Device 1, IoTScope identified 4 vulnerabilities, achieving a recall of 28.6%. FALCONSCOPE, however, correctly identified 13 vulnerabilities on the same device, reaching a 92% recall. Therefore, by achieving both broader coverage and higher accuracy in verification, FALCONSCOPE enables the discovery of far more real-world vulnerabilities than the current state-of-the-art.

Efficiency. In terms of efficiency, Table 2 similarly demonstrates FALCONSCOPE’s remarkable advantage. FALCONSCOPE completed its analysis with an average duration of merely 472.73 seconds per device, representing just 3.6% of the time required by IoTScope (13,095.45 seconds, roughly 3.6 hours)—an approximate 28-fold improvement. Moreover, this efficiency is achieved with surgical precision. FALCONSCOPE sent an average of only 309.18 probing requests per device, reducing the request volume by approximately 97.6% compared to IoTScope’s average of 12,743.18. FALCONSCOPE’s substantial efficiency gains stem from its carefully designed two-stage strategy. Initially, its precise static analysis (Section 3.1) effectively identifies high-probability interface candidates, avoiding the blind enumeration that plagues IoTScope. Subsequently, its intelligent, LLM-guided dynamic probing ensures that only meaningful requests are sent. Conversely, IoTScope’s exhaustive URL-generation approach produced an excessive number of candidate requests (totaling 140,175), most of which proved invalid, incurring substantial computational overhead and wasted analytical resources.

4.3 Effectiveness of Interface Enumerator

To evaluate the accuracy and coverage of FALCONSCOPE’s Interface Enumerator, we compared it with SaTC [4], a recent influential work in IoT firmware analysis. SaTC leverages the observation that IoT device front-end and back-end components often share common keywords, allowing it to precisely locate back-end functions. Consequently, it can also detect Type-1 back-end logic as defined in Section 3.1. We conducted comparative experiments on devices from four vendors. As shown in Table 3, FALCONSCOPE significantly outperforms SaTC. In terms of precision, FALCONSCOPE achieved a perfect 100% accuracy, where all enumerated interfaces were confirmed valid. In contrast, SaTC generated numerous false positives on some devices (e.g., only 103 valid out of 502 enumerated on DIR878), resulting in substantial unnecessary overhead for dynamic testing. Regarding coverage, FALCONSCOPE discovered a much larger number of valid interfaces than SaTC, owing to its more comprehensive static analysis strategy. While SaTC primarily depends on shared keywords between the front-end and back-end, thereby missing interfaces not explicitly referenced in front-end code, FALCONSCOPE constructs a more complete routing map through Algorithm 1. Notably, FALCONSCOPE’s set of function-type interfaces fully subsumes those identified by SaTC, and it can further enumerate file or script-type interfaces.

4.4 Effectiveness of LLM-driven Components

To quantitatively assess the impact of the two core LLM-powered components in our design, we conducted an ablation study involving three configurations: *FS-R*, which disables the LLM-based request generator and replaces it with a traditional random fuzzer;

FS-B, which substitutes the LLM-based bug oracle with a conventional keyword-matching oracle; and *FS*, the complete, unmodified FALCONSCOPE system. The results, summarized in Table 4, demonstrate the critical role of both LLM-driven components. Comparing *FS* with *FS-R* reveals that the latter failed to detect any vulnerabilities across all evaluated devices (0% recall), underscoring the inadequacy of random fuzzing for logic bug discovery. Without semantic understanding of back-end behavior, randomly generated inputs cannot satisfy the complex syntactic and semantic conditions required to activate deep application logic. This finding establishes that LLM-driven request generation is not an auxiliary optimization but an essential prerequisite for uncovering logic vulnerabilities. Similarly, comparison between *FS* and *FS-B* highlights the necessity of the LLM-based bug oracle. While *FS-B* identified a limited subset of vulnerabilities (e.g., 12/13 on DIR823G), its recall dropped sharply on devices with complex or irregular response formats (to 40%, 10%, and even 0%). In contrast, FALCONSCOPE’s LLM-based oracle effectively interprets heterogeneous response semantics, enabling robust detection performance across diverse firmware ecosystems.

Table 3: Comparison between SaTC and FalconScope. Bold indicates matches to the pattern shown in Fig. 3(a)

Compare		DIR878	AC18	DIR816	R6300
SaTC	#TP	103	101	40	129
	#Enum	502	116	45	137
FalconScope	#TP	376(129)	279(188)	380(302)	793(394)
	#Enum	376	279	380	793

Table 4: Comparison of different FALCON variants.

Method	A3700R		FH1202		DIR-816		DIR-823G		DIR-618	
	V/G	%	V/G	%	V/G	%	V/G	%	V/G	%
FS-R	0	0	0	0	0/10	0	0	0	0	0
FS-B	4/10	40%	1/9	11.1%	0/10	0	12/13	92.3%	0/10	0
FS	10/10	100%	9/9	100%	10/10	100%	12/13	92.3%	10/10	100%

4.5 Analysis of Payload Format and RUI Location Consistency

A core insight guiding FALCONSCOPE’s request generation strategy is that the communication paradigm—encompassing both the payload format and the RUI embedding location—is highly consistent within a specific device model. To validate this hypothesis, we analyzed the consistency of these two factors across a set of 58 confirmed vulnerabilities discovered by FALCONSCOPE. As shown in Table 7, the empirical evidence is definitive: both the payload format and the RUI embedding location exhibit 100% consistency across all tested devices. For any given device model, all identified vulnerabilities were triggered using the same payload format (e.g., XML) and by placing the RUI in the exact same location (e.g., the SOAPAction header or a specific URI parameter like topicurl). This powerful finding provides a robust justification for our use of a few-shot context in the LLM-driven request synthesis phase. By capturing just a few sample requests, FALCONSCOPE can reliably infer the entire, standardized communication paradigm for a device.

5 Discussion and Limitations

FALCONSCOPE primarily targets the interface paradigms described in Section 3.1. Its core pipeline assumes that firmware is extractable and that the embedded web-service binaries can be successfully decompiled into usable pseudocode for subsequent analysis. Sophisticated anti-analysis measures (e.g., heavy obfuscation or encryption) may render firmware non-extractable, or cause decompilation to fail or produce low-fidelity code. In such cases, LLM-based request generation is severely hindered, which limits the applicability of our tool to heavily protected firmware. Addressing these advanced defensive measures is a key direction for future work.

6 Related Work

Broken Access Control. Research efforts on detecting hidden interfaces remain inadequate. IoTScope [37] attempts to uncover unauthorized hidden interfaces through exhaustive URL enumeration but overlooks the semantic information of requests and fails to identify interfaces involving function execution. Several studies have focused on broken access control in mobile and cloud services [1, 12–14, 30, 42, 45]. AuthScope [47] concentrates on identifying vulnerable access control implementations on the server side of mobile applications. LeakScope [46] detects data leakage in cloud backend services by analyzing mobile applications.

Vulnerability Discovery in IoT Devices. Prior work [7, 28, 31, 32, 43, 44] employs a variety of approaches to discover vulnerabilities in IoT devices. Static approaches such as SaTC [4] builds on the insight that variable names shared between the frontend and backend can be used to pinpoint input entry points in backend binaries. Dynamic approaches such as SRFuzzer [41] automate the discovery of various vulnerabilities in SOHO routers by modeling communication formats, while LABRADOR [20] introduces a response-guided directed fuzzing strategy to improve fuzzing efficiency. To support dynamic testing, some research efforts [3, 9, 15, 18, 40] are dedicated to simulating IoT devices. With the popularity of LLMs, many works [17, 22, 23, 39] use LLMs for IoT vulnerability detection.

7 Conclusion

In this paper, we proposed FALCONSCOPE, which to our knowledge is the first system that unifies precise firmware routing modeling with LLM-driven semantic request generation and response validation for detecting unauthorized hidden interfaces. Compared with existing approaches, FALCONSCOPE’s Interface Enumerator systematically enumerates RUI-to-handler mappings, while its Semantically-Enhanced Request Generator and Unauthorized Access Verifier enable semantically valid probing and reliable unauthorized-access confirmation. In evaluation on 11 real-world IoT devices, FALCONSCOPE identified 620 unauthorized hidden interfaces (103× more than IoTScope), achieved 98.6% average recall, and reduced probing requests by approximately 97.6%, while consuming only 3.6% of IoTScope’s analysis time. Following responsible disclosure, 50 issues have been assigned CVE IDs.

Acknowledgments

The research is supported in part by the National Natural Science Foundation of China (No.62202465), the Beijing Key Laboratory of Network Security Protection Technology (No.2022YFB3103900).

References

- [1] Safwa Ameer, James Benson, and Ravi Sandhu. 2022. Hybrid approaches (ABAC and RBAC) toward secure access control in smart home IoT. *IEEE transactions on dependable and secure computing* 20, 5 (2022), 4032–4051.
- [2] IoT Analytics. 2024. Number of connected IoT devices to reach 40 billion by 2030. <https://iot-analytics.com/number-connected-iot-devices>.
- [3] Daming D Chen, Maverick Woo, David Brumley, and Manuel Egele. 2016. Towards automated dynamic analysis for linux-based embedded firmware.. In *NDSS*, Vol. 1. 1–1.
- [4] Libo Chen, Yanhao Wang, Quanpu Cai, Yunfan Zhan, Hong Hu, Jiaqi Linghu, Qinsheng Hou, Chao Zhang, Haixin Duan, and Zhi Xue. 2021. Sharing more and checking less: Leveraging common input keywords to detect bugs in embedded systems. In *30th USENIX Security Symposium*.
- [5] Django Software Foundation. 2025. *Django*. <https://github.com/django/django>
- [6] Expressjs. 2025. *Express*. <https://github.com/expressjs/express>
- [7] Xiaotao Feng, Ruoxi Sun, Xiaogang Zhu, Minhui Xue, Sheng Wen, Dongxi Liu, Surya Nepal, and Yang Xiang. 2021. Snipuzz: Black-box fuzzing of iot firmware via message snippet inference. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*. 337–350.
- [8] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2024. Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [9] Eric Gustafson, Marius Muench, Chad Spensky, Nilo Redini, Aravind Machiry, Yanick Fratantonio, Davide Balzarotti, Aurélien Francillon, Yung Ryn Choe, Christophe Kruegel, et al. 2019. Toward the analysis of embedded firmware through automated re-hosting. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)*. 135–150.
- [10] Hex-Rays. 2025. *IDA Pro: The Industry Standard Disassembler and Debugger*. <https://hex-rays.com/ida-pro/>
- [11] Ridhi Jain, Nicole Gervasoni, Mthandazo Ndhlovu, and Sanjay Rawat. 2023. A code centric evaluation of c/c++ vulnerability datasets for deep learning based vulnerability detection techniques. In *Proceedings of the 16th Innovations in Software Engineering Conference*. 1–10.
- [12] Yan Jia, Luyi Xing, Yuhang Mao, Dongfang Zhao, XiaoFeng Wang, Shangru Zhao, and Yuting Zhang. 2020. Burglars' iot paradise: Understanding and mitigating security risks of general messaging protocols on iot clouds. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 465–481.
- [13] Yunhan Jack Jia, Qi Alfred Chen, Shiqi Wang, Amir Rahmati, Earlene Fernandes, Zhuoqing Morley Mao, Atul Prakash, and SJ University. 2017. ContextIoT: Towards providing contextual integrity to appified IoT platforms.. In *ndss*, Vol. 2. San Diego, 2–2.
- [14] Ze Jin, Luyi Xing, Yiwei Fang, Yan Jia, Bin Yuan, and Qixu Liu. 2022. P-verifier: Understanding and mitigating security risks in cloud-based IoT access policies. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*.
- [15] Evan Johnson, Maxwell Bland, YiFei Zhu, Joshua Mason, Stephen Checkoway, Stefan Savage, and Kirill Levchenko. 2021. Jetset: Targeted firmware rehosting for embedded systems. In *30th USENIX Security Symposium (USENIX Security 21)*. 321–338.
- [16] Nikitas Karanikolas, Eirini Manga, Nikoleta Samaridi, Eleni Tousidou, and Michael Vassilakopoulos. 2023. Large language models versus natural language understanding and generation. In *Proceedings of the 27th Pan-Hellenic Conference on Progress in Computing and Informatics*. 278–290.
- [17] Avishree Khare, Saikat Dutta, Ziyang Li, Alaia Solko-Breslin, Rajeev Alur, and Mayur Naik. 2023. Understanding the effectiveness of large language models in detecting security vulnerabilities. *arXiv preprint arXiv:2311.16169* (2023).
- [18] Mingun Kim, Dongkwan Kim, Eunsoo Kim, Suryeon Kim, Yeongjin Jang, and Yongdae Kim. 2020. Firmae: Towards large-scale emulation of iot firmware for dynamic analysis. In *Proceedings of the 36th Annual Computer Security Applications Conference*. 733–745.
- [19] LangChain AI. 2025. *langchain*. <https://github.com/langchain-ai/langchain>
- [20] Hangtian Liu, Shuitao Gan, Chao Zhang, Zicong Gao, Hongqi Zhang, Xiangzhi Wang, and Guangming Gao. 2024. Labrador: response guided directed fuzzing for black-box IoT devices. In *IEEE Symposium on Security and Privacy*.
- [21] Hangtian Liu, Lei Zheng, Shuitao Gan, Chao Zhang, Zicong Gao, Hongqi Zhang, Yishun Zeng, Zhiyuan Jiang, and Jiahai Yang. 2025. EAGLEYE: Exposing Hidden Web Interfaces in IoT Devices via Routing Analysis. In *32nd Annual Network and Distributed System Security Symposium*.
- [22] Puzhuo Liu, Chengnian Sun, Yaowen Zheng, Xuan Feng, Chuan Qin, Yuncheng Wang, Zhi Li, and Limin Sun. 2023. Harnessing the power of llm to support binary taint analysis. *arXiv preprint arXiv:2310.08275* (2023).
- [23] Puzhuo Liu, Chengnian Sun, Yaowen Zheng, Xuan Feng, Chuan Qin, Yuncheng Wang, Zhenyang Xu, Zhi Li, Peng Di, Yu Jiang, et al. 2025. Llm-powered static binary taint analysis. *ACM Transactions on Software Engineering and Methodology* (2025).
- [24] National Vulnerability Database. 2024. CVE-2024-13030. <https://nvd.nist.gov/vuln/detail/CVE-2024-13030>.
- [25] National Vulnerability Database. 2025. Vulnerability Metrics (CVSS). <https://nvd.nist.gov/vuln-metrics/cvss>.
- [26] OpenAI. 2025. GPT-4 Models - OpenAI API. <https://platform.openai.com/docs/models/gpt-4>.
- [27] OWASP. 2021. OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten>. Discusses Broken Access Control (A01).
- [28] Nilo Redini, Aravind Machiry, Ruoyu Wang, Chad Spensky, Andrea Continella, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2020. Karonte: Detecting insecure multi-binary interactions in embedded firmware. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1544–1561.
- [29] ReFirm Labs. 2025. *binwalk*. <https://github.com/ReFirmLabs/binwalk>
- [30] Roei Schuster, Vitaly Shmatikov, and Eran Tromer. 2018. Situational access control in the internet of things. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 1056–1073.
- [31] Prashast Srivastava, Hui Peng, Jiahao Li, Hamed Okhravi, Howard Shrobe, and Mathias Payer. 2019. Firmfuzz: Automated iot firmware introspection and analysis. In *Proceedings of the 2nd International ACM Workshop on Security and Privacy for the Internet-of-Things*.
- [32] Hui Jun Tay, Kyle Zeng, Jayakrishna Menon Vadayath, Arvind S Raj, Audrey Dutcher, Tejesh Reddy, Wil Gibbs, Zion Leonahenahe Basque, Fangzhou Dong, Zack Smith, et al. 2023. Greenhouse: {Single-Service} rehosting of {Linux-Based} firmware binaries in {User-Space} emulation. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5791–5808.
- [33] The Pallets Projects. 2025. *Flask*. <https://github.com/pallets/flask>
- [34] ThreatPost. 2020. Half of IoT devices are vulnerable to severe attacks. <https://threatpost.com/half-iot-devices-vulnerable-severe-attacks/153609>.
- [35] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*. 1–7.
- [36] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* (2022).
- [37] Wei Xie, Jiongqi Chen, Zhenhua Wang, Chao Feng, Enze Wang, Yifei Gao, Baosheng Wang, and Kai Lu. 2022. Game of hide-and-seek: Exposing hidden interfaces in embedded web applications of iot devices. In *Proceedings of the ACM Web Conference 2022*.
- [38] Jia-Yu Yao, Kun-Peng Ning, Zhen-Hui Liu, Mu-Nan Ning, Yu-Yang Liu, and Li Yuan. 2023. Llm lies: Hallucinations are not bugs, but features as adversarial examples. *arXiv preprint arXiv:2310.01469* (2023).
- [39] Junjian Ye, Xincheng Fei, Xavier de Carné de Carnavalet, Lianying Zhao, Lifa Wu, and Mengyuan Zhang. 2024. Detecting command injection vulnerabilities in Linux-based embedded firmware with LLM-based taint analysis of library functions. *Computers & Security* 144 (2024), 103971.
- [40] Jonas Zaddach, Luca Bruno, Aurelien Francillon, Davide Balzarotti, et al. 2014. AVATAR: A Framework to Support Dynamic Security Analysis of Embedded Systems' Firmwares.. In *NDSS*, Vol. 14. 1–16.
- [41] Yu Zhang, Wei Huo, Kunpeng Jian, Ji Shi, Haojiang Lu, Longquan Liu, Chen Wang, Dandan Sun, Chao Zhang, and Baoxu Liu. 2019. SRFuzzer: An automatic fuzzing framework for physical SOHO router devices to discover multi-type vulnerabilities. In *Proceedings of the 35th annual computer security applications conference*.
- [42] Zehui Zhang, Futai Zou, Jianan Hong, Libo Chen, and Ping Yi. 2024. Detection and Analysis of Broken Access Control Vulnerabilities in App-Cloud Interaction in IoT. *IEEE Internet of Things Journal* (2024).
- [43] Jiaxu Zhao, Yuekang Li, Yanyan Zou, Zhaohui Liang, Yang Xiao, Yeting Li, Bingwei Peng, Nanyu Zhong, Xinyi Wang, Wei Wang, et al. 2024. Leveraging semantic relations in code and data to enhance taint analysis of embedded systems. In *33rd USENIX Security Symposium*.
- [44] Yaowen Zheng, Ali Davanian, Heng Yin, Chengyu Song, Hongsong Zhu, and Limin Sun. 2019. {FIRM-AFL}:{High-Throughput} greybox fuzzing of {IoT} firmware via augmented process emulation. In *28th USENIX Security Symposium (USENIX Security 19)*. 1099–1114.
- [45] Chaoshun Zuo and Zhiqiang Lin. 2017. Smartgen: Exposing server urls of mobile apps with selective symbolic execution. In *Proceedings of the 26th International Conference on World Wide Web*.
- [46] Chaoshun Zuo, Zhiqiang Lin, and Yinqian Zhang. 2019. Why does your data leak? uncovering the data leakage in cloud from mobile apps. In *IEEE Symposium on Security and Privacy*.
- [47] Chaoshun Zuo, Qingchuan Zhao, and Zhiqiang Lin. 2017. Authscope: Towards automatic discovery of vulnerable authorizations in online services. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*.

A Appendix

A.1 Interface Enumeration Algorithm

Algorithm 1: Interface Enumeration Algorithm

```

Input: Backend Binary  $B$ , Filesystem Root  $FS$ , Frontend URI set  $FEURIs$ 
Output: Set of identified interfaces  $Interfaces$ : (Identifier, Type, Handler)
/* Initialization */
 $Ifaces, Proc \leftarrow \emptyset, \emptyset$ ;
 $ScrExts \leftarrow \{.sh, \dots\}, StatExts \leftarrow \{.html, \dots\}$ ;
/* Phase 1: Scan filesystem for file interfaces */
for  $file \in RecScan(FS)$  do
  if  $file.ext \in ScrExts$  then
     $Ifaces \leftarrow Ifaces \cup \{(file.path, SCRIPT, file.path)\}$ ;
  else if  $file.ext \in StatExts$  then
     $Ifaces \leftarrow Ifaces \cup \{(file.path, FILE, file.path)\}$ ;
 $Ifaces \leftarrow LLMFilter(Ifaces)$ ;
/* Phase 2: Scan bin for URI references and infer interfaces */
for  $uri \in FEURIs$  do
  for  $xref \in FindRefs(B, uri)$  do
    if  $InTextSec(xref) \ \&\& \ IsRegParam(uri)$  then
      // URI used as param -> dynamic registration
       $initFunc \leftarrow GetCaller(B, xref)$ ;
      if  $initFunc \notin Proc$  then
         $Ifaces \leftarrow Ifaces \cup ExtractRegs(initFunc)$ ;
         $Proc \leftarrow Proc \cup \{initFunc\}$ ;
    else if  $InDataSec(xref)$  then
      // URI in data -> static map
       $region \leftarrow GetRegion(xref)$ ;
      if  $region \notin Proc$  then
         $Ifaces \leftarrow Ifaces \cup ExtractMaps(B, region)$ ;
         $Proc \leftarrow Proc \cup \{region\}$ ;
return  $Ifaces$ ;
```

A.2 Empirical Analysis

Table 5: Prevalence of Identified Routing Patterns Across a Diverse Set of IoT Firmware. We conducted a large-scale empirical analysis on firmware from 10 vendors and 120 samples, covering multiple categories of IoT devices, including Routers, APs, Cameras, Switches, Range Extenders, Bridges.

Vendor	Samples	Type 1		Type 2	
		Match	%	Match	%
DLINK	13	11	84.6%	13	100%
Tenda	19	19	100%	19	100%
TOTOLINK	2	2	100%	2	100%
NETGEAR	27	27	100%	27	100%
Asus	18	18	100%	18	100%
Trendnet	13	12	92.3%	13	100%
H3C	13	12	92.3%	13	100%
TP-Link	8	8	100%	8	100%
Linksys	6	6	100%	6	100%
Fortinet	1	1	100%	1	100%
Total	120	116	96.70%	120	100%

For each sample, we used Binwalk to extract the filesystem and combined it with IDA to analyze the web server binaries, then performed manual inspection to determine whether it follows the Type-1/Type-2 routing patterns.

A.3 Case Studies

We present two representative case studies. Each illustrates a distinct type of hidden interface vulnerability discovered during our evaluation in Table 1, and all have been assigned CVE identifiers.

Case 1: Unauthorized Remote Control via Backend Functions (CVE-2024-13030) In a widely deployed consumer router, FALCONSCOPE uncovered a critical vulnerability allowing unauthenticated attackers to remotely control essential device functions. The Interface Enumerator first extracted a comprehensive static routing map from the device’s firmware image, revealing multiple sensitive Routing Unique Identifiers (RUIs) tied to privileged operations, including SetFirewallSettings and SetDMZSettings. The Request Generator then targeted the SetFirewallSettings RUI by analyzing its decompiled handler code, enabling the LLM to construct a semantically valid POST request designed to alter firewall rules. When sent without any authentication credentials, this request prompted a generic response: "SettingResult OK". Critically, LLM-based Bug Oracle correctly interpreted this seemingly benign message not as a simple string, but as a contextual confirmation of a successful, unauthorized configuration change, thereby validating the vulnerability.

Case 2: Sensitive Data Leakage via Script Execution (CVE-2024-7156) In another IoT device, FALCONSCOPE identified an exposed server-side script that led to a severe information disclosure vulnerability. The system identified an executable script, export_settings.sh, located in a web-accessible directory that, though likely intended for administrative backups, lacked proper access controls. An unauthenticated HTTP GET request sent by FALCONSCOPE successfully triggered the script’s execution, prompting the device to respond by returning its complete configuration file as a plain-text attachment. The LLM-based Bug Oracle parsed the returned data, identified sensitive information such as credentials and network settings, and confirmed a high-severity vulnerability that enables unauthorized access to critical system information.

A.4 Request and Response Samples

```

POST /HNAP1/ HTTP/1.1
Host: 192.168.0.1
Content-Type: text/xml; charset=utf-8
SOAPAction: "http://purenetworks.com/HNAP1/SetDMZSettings"

<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope>
  <soap:Body>
    <SetDMZSettings xmlns="http://purenetworks.com/HNAP1/">
      <Enabled>true</Enabled>
      <IPAddress>192.168.0.251</IPAddress>
    </SetDMZSettings>
  </soap:Body>
</soap:Envelope>
```

Listing 1: An example of an unauthenticated request crafted by FALCONSCOPE to set the DMZ configuration. Note the absence of any Cookie or other authentication token.

```
HTTP/1.0 200 Data follows
Content-Type: text/html

<?xml version="1.0" encoding="utf-8"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV=
"http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <SetDMZSettingsResponse
      xmlns="http://purenetworks.com/HNAP1/">
      <SetDMZSettingsResult>
        OK
      </SetDMZSettingsResult>
    </SetDMZSettingsResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Listing 2: The corresponding success response from the device. The "OK" result within the <SetDMZSettingsResult> tag confirms that the unauthenticated request was successfully processed.

A.5 Prompts

The Crafted LLM Prompt For Request Generation

Role: System

You are an expert IoT security researcher. Task: analyze provided firmware code snippets and network request samples to produce one precise HTTP request that is syntactically and semantically correct to trigger the specified backend handler. Follow the user's instructions and Chain-of-Thought exactly. Output only the raw HTTP request with no additional text, explanations, or formatting.

INPUT DATA:

- (1) [Few-shot legitimate request samples] (structural reference).
- (2) [PASTE TARGET RUI HERE] e.g., "SetDDNSSettings"
- (3) [PASTE Decompiled C pseudocode of the handler]

TASK (follow the Chain-of-Thought precisely):

- (1) Analyze structure (from samples): Infer HTTP method, end-point/RUI location, headers, and payload format.
- (2) Analyze semantics (from pseudocode): Identify required parameters, expected types/values, and overall handler behavior. Decide whether the handler can leak sensitive router data (passwords, config, etc.), or modify device state or configuration (rewrite settings, enable/disable services, reboot, etc.).
- (3) If the handler can modify device state, construct a formatted HTTP POST that includes the necessary parameters.
- (4) Synthesize the full HTTP request: Produce a complete request (request line, headers, and body) consistent with the inferred structure and semantics. remove authentication — omit the Cookie header.

Role: User

- (1) Few-Shot Samples: [Placeholder]
- (2) Target RUI: [Placeholder]
- (3) Handler Pseudocode: [Placeholder]

Figure 4: The Crafted LLM Prompt For Request Generation.

The Crafted LLM Prompt For Bug Oracle

Role: System

You are an expert IoT security analyst. Your task is to determine if an unauthenticated HTTP request successfully exploited a Broken Access Control vulnerability. Analyze the device's response in the context of the original request's intent. Your final output must be a structured JSON object containing your verdict and the evidence.

Role: User

CONTEXT:

- (1) Original HTTP Request: [Placeholder for the request sent]
- (2) Device's HTTP Response: [Placeholder for the response received]

TASK:

Analyze the device's response. Determine if the response contains clear evidence of a successful operation. Evidence of success includes, but is not limited to:

- Unauthorized State Setting: A response body that explicitly confirms a state change (e.g., a JSON/XML echo of the new settings).
- Unauthorized Information Leakage: A response body containing sensitive data (e.g., system command output, configuration files, private keys).

Provide your final verdict in the following JSON format ONLY:

```
{
  "verdict": "SUCCESS" | "FAILURE" | "UNCERTAIN",
  "reason": "A brief explanation for your verdict.",
  "evidence": "The specific text from the response that supports your reason."
}
```

Figure 5: The Crafted LLM Prompt For Bug Oracle.

```
{
  "SetPasswdSettings": "sub_43D518",
  "GetLocalMacAddress": "sub_446C30",
  "SetIgnoreWizardConfig": "sub_435D4C",
  "GetNTPServerSettings": "sub_43DA48",
  "SetNTPServerSettings": "sub_43E0AC",
  "GetDeviceDomainName": "sub_43E51C",
  "SetPPPoEServerSettings": "sub_4365C0",
  "GetPPPoEServerStatus": "sub_436048",
  "GetLanWanConflictInfo": "sub_452160",
  "...": "..."
}
```

Listing 3: A partial RUI-to-handler mapping table extracted by FALCONSCOPE from the DIR-823G firmware. Each entry maps a Routing Unique Identifier (RUI) string to the corresponding backend handler function.

A.6 CVE Lists

Table 6: Detailed CVE List detected by FalconScope (confirmed as of this writing). For the bug type, BAC represents Broken Access Control, ID represents information disclosure. Body represents the payload format needed to trigger the vulnerability. Service represents the service where the vulnerability occurs, Mult* indicates that the CVE affects multiple services. 2/3* indicates that Some CVE identifiers assigned to devices 2 and 3 were partially consolidated by the CVE Numbering Authority.

Device ID	Type	Body	Service	CVE-ID
1	BAC	xml	Mult* DDNS UPnP	CVE-2024-13030 CVE-2025-2359 CVE-2025-2360
2/3*	BAC	key-value	HTTP DDNS	CVE-2025-2546, CVE-2025-2547, CVE-2025-2548, CVE-2025-2549, CVE-2025-2551, CVE-2025-2552, CVE-2025-2553 CVE-2025-2550
4	BAC	key-value	HTTP DHCP QoS DMZ DDNS	CVE-2024-57676, CVE-2024-57677, CVE-2024-57678, CVE-2024-57679, CVE-2024-57680, CVE-2024-57681, CVE-2024-57682, CVE-2024-57683, CVE-2024-13103, CVE-2024-13104, CVE-2024-13107, CVE-2024-13108 CVE-2024-13105 CVE-2024-13106 CVE-2024-57684 CVE-2024-13102
5	ID	None	HTTP	CVE-2025-0481
6	CSRF	key-value	HTTP	CVE-2024-2559, CVE-2024-2560
	ID	None	HTTP	CVE-2025-2993
7	BAC	key-value	HTTP QoS DDNS DMZ	CVE-2025-3237, CVE-2025-2989, CVE-2025-2990, CVE-2025-2991, CVE-2025-2992, CVE-2025-2995 CVE-2025-2994 CVE-2025-2996 CVE-2025-3236
10	ID	None	HTTP	CVE-2024-7156
	BAC	json	HTTP QoS DDNS UPnP	CVE-2024-7154, CVE-2025-3663, CVE-2025-3664, CVE-2025-3668, CVE-2025-3674, CVE-2025-3675 CVE-2025-3665 CVE-2025-3666 CVE-2025-3667

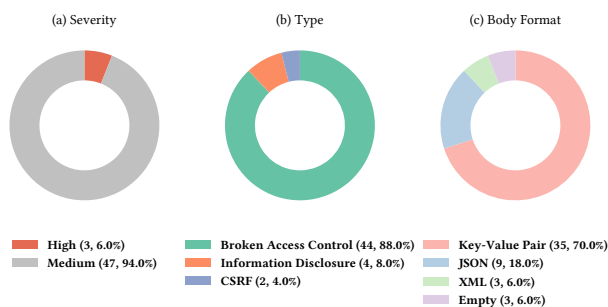


Figure 6: Distribution characteristics of 50 CVEs found by FALCONSCOPE: Severity, Vulnerability Type, and Format.

Severity of Hidden Interfaces. We evaluated confirmed vulnerabilities using the CVSS [25]. As shown in Figure 6(a), among the 50 CVEs, 3 were rated as high severity and 47 as medium severity. All vulnerabilities could be remotely exploited through carefully crafted requests, resulting in unauthorized device modifications or sensitive information disclosure, underscoring the significant real-world threat posed by hidden interfaces as an attack vector.

Ethical Considerations. Our research was conducted in strict adherence to ethical guidelines for security research. All IoT devices used in our evaluation were legally purchased and owned by the research team. The experiments were performed in an isolated laboratory network to prevent any impact on public networks or other users. We have followed a coordinated vulnerability disclosure (CVD) process by responsibly reporting all discovered vulnerabilities to the respective vendors. Furthermore, we have submitted our findings to the Common Vulnerabilities and Exposures (CVE) program, and we are actively collaborating with vendors to ensure the reported issues are addressed.

Table 7: Analysis of Payload Format and RUI Location.

Device	Payload		RUI	
	Format	#Match/#Total	Location	#Match/#Total
DIR-823G	XML	10 / 10 (100%)	SOAPAction	10 / 10 (100%)
DIR-618	K-V	8 / 8 (100%)	URI Path	8 / 8 (100%)
DIR-605L	K-V	8 / 8 (100%)	URI Path	8 / 8 (100%)
DIR-816	K-V	16 / 16 (100%)	URI Path	16 / 16 (100%)
FH1202	K-V	9 / 9 (100%)	URI Path	9 / 9 (100%)
A3700R	JSON	10 / 10 (100%)	topicurl	10 / 10 (100%)